05_models_svm_ann

Contents

1 05 Support Vector Machine and Artificial Neural Network **1**
1.1 1. Libraries . . . 1
1.2 2. Constants . . . 1
1.3 3. Load Data and Reproduce Split . . . 2
1.4 4. Metrics Helper . . . 3
1.5 5. Support Vector Machine (RBF Kernel) . . . 4
1.6 6. Artificial Neural Network (nnet — Single Hidden Layer) . . . 6
1.7 7. Results Summary . . . 9
1.8 8. Save Outputs . . . 10

1 05 Support Vector Machine and Artificial Neural Network

Author: Aaron **Purpose:** Estimate Support Vector Machine (SVM, RBF kernel) and Artificial Neural Network (ANN) models across all three prediction horizons (1-day, 3-day, 5-day). Both models use feature normalisation via `step_normalize()` inside the tidymodels workflow, ensuring the scaling transform is fitted on the training fold only and never contaminates the assessment fold or test set.

Depends on: `data/processed/features.csv` (output of `02_feature_engineering.Rmd`)

Note on ANN architecture: The `nnet` package (as specified in the rubric) implements a single hidden-layer perceptron with logistic activation and L2 weight decay (`penalty`). It does **not** support ReLU activation or dropout. This is a constraint of the `nnet` package. The regularisation via weight decay serves a function analogous to dropout by penalising large weights and reducing overfitting.

1.1 1. Libraries

```r
library(tidymodels)    # parsnip, recipes, tune, rsample
library(kernlab)       # SVM engine
library(nnet)          # single-layer ANN engine
library(kableExtra)    # LaTeX table export
library(tidyverse)
library(here)
```

1.2 2. Constants

```r
# Reproducibility
RANDOM_SEED <- 42L

# Data split - must mirror 02_feature_engineering.Rmd Section 10
TRAIN_PROP  <- 0.80
```

```r
# Rolling-origin CV
CV_INITIAL  <- 200L   # minimum training window
CV_ASSESS   <- 60L    # assessment window (~one trading quarter)
CV_SKIP     <- 100L   # gives ~13 slices on 80% of 1850 obs; kernlab SVM is slow, increase to speed up

# SVM
SVM_GRID_SIZE <- 15L  # Latin hypercube grid points for cost x rbf_sigma

# ANN
ANN_EPOCHS     <- 200L
ANN_GRID_SIZE  <- 15L  # Latin hypercube grid points for hidden_units x penalty
ANN_HIDDEN_MIN <- 3L
ANN_HIDDEN_MAX <- 15L  # cap avoids overly large networks for n~1400 training obs

# Column definitions
TARGET_COLS  <- c("target_1d", "target_3d", "target_5d")
HORIZON_LABS <- c("1d",        "3d",        "5d")

PREDICTOR_COLS <- c(
  # contemporaneous returns (known at time t)
  "r_bpi", "r_bdo", "r_psei", "r_fx", "r_us10y",
  # trend
  "ma5", "ma10", "ma20", "ema12", "ema26", "ma_ratio",
  # momentum
  "lag1", "lag2", "lag3", "mom5", "mom10", "rsi14", "macd", "macd_signal",
  # volatility
  "vol5", "vol10", "vol20", "vol_ratio",
  # cross-asset
  "lag_psei",
  # engineered transformations
  "interact", "vol5_sq",
  # regime
  "hi_vol"
)
```

1.3 3. Load Data and Reproduce Split

```r
features <- read_csv(
  here("data", "processed", "features.csv"),
  show_col_types = FALSE
)

n_total <- nrow(features)
n_train <- floor(TRAIN_PROP * n_total)
n_test  <- n_total - n_train

train_df <- features[seq_len(n_train), ]
test_df  <- features[(n_train + 1L):n_total, ]

cat("Total clean observations:", n_total, "\n")
```

```
## Total clean observations: 2623
```

```r
cat("Training set            :", n_train,
    "| up to", format(train_df$date[nrow(train_df)], "%Y-%m-%d"), "\n")
```

```
## Training set            : 2098 | up to 2024-11-01
```

```r
cat("Test set                :", n_test,
    "| from", format(test_df$date[1], "%Y-%m-%d"), "\n")
```

```
## Test set                : 525 | from 2024-11-02
```

1.4 4. Metrics Helper

```r
compute_metrics <- function(test_actual, test_pred, train_actual, train_pred, naive_pred, model, horizon
  test_ss_res   <- sum((test_actual - test_pred)^2)
  test_ss_total <- sum((test_actual - mean(test_actual))^2)
  train_ss_res  <- sum((train_actual - train_pred)^2)
  train_ss_total <- sum((train_actual - mean(train_actual))^2)

  tibble(
    Model      = model,
    Horizon    = horizon,
    RMSE       = sqrt(mean((test_actual - test_pred)^2)),
    MAE        = mean(abs(test_actual - test_pred)),
    R2         = 1 - test_ss_res / test_ss_total,
    DA         = mean(sign(test_pred) == sign(test_actual)),
    Naive_DA   = mean(sign(naive_pred) == sign(test_actual)),
    Train_RMSE = sqrt(mean((train_actual - train_pred)^2)),
    Train_MAE  = mean(abs(train_actual - train_pred)),
    Train_R2   = 1 - train_ss_res / train_ss_total,
    Train_DA   = mean(sign(train_pred) == sign(train_actual))
  )
}

# QVRS Economist Theme
theme_quant <- function() {
  theme_minimal() +
    theme(
      text = element_text(color = "#0D0D0D"),
      plot.title = element_text(face = "bold", size = 14, margin = margin(b = 5)),
      plot.subtitle = element_text(size = 11, color = "#333333", margin = margin(b = 10)),
      plot.caption = element_text(size = 8, color = "#595959", hjust = 0, margin = margin(t = 10)),
      axis.title = element_text(size = 10, face = "bold"),
      axis.text = element_text(size = 9),
      panel.grid.minor = element_blank(),
      panel.grid.major = element_line(color = "#F2F2F2", linewidth = 0.3),
      legend.position = "bottom",
      axis.ticks = element_blank()
    )
}
```

1.5 5. Support Vector Machine (RBF Kernel)

SVM requires all numeric features to be on a comparable scale. The recipe includes `step_normalize(all_numeric_predictors)` which centres and scales each predictor to zero mean and unit variance. Critically, `prep()` is called once per CV fold inside the tidymodels workflow — the normalisation parameters are derived only from the training portion of each fold and are never exposed to the assessment or test data.

We tune two kernel hyperparameters:

Parameter	Controls
`cost`	Regularisation; lower values = wider margin, more slack
`rbf_sigma`	RBF kernel bandwidth; higher values = tighter decision boundary

```r
set.seed(RANDOM_SEED)

svm_metrics_list <- vector("list", length(TARGET_COLS))

for (h in seq_along(TARGET_COLS)) {

  target_col    <- TARGET_COLS[h]
  horizon_label <- HORIZON_LABS[h]

  cat("\nSVM |", toupper(horizon_label), "horizon\n")

  train_h <- train_df %>% select(all_of(c(target_col, PREDICTOR_COLS)))
  test_h  <- test_df  %>% select(all_of(c(target_col, PREDICTOR_COLS)))

  cv_h <- rolling_origin(
    data       = train_h,
    initial    = CV_INITIAL,
    assess     = CV_ASSESS,
    skip       = CV_SKIP,
    cumulative = TRUE
  )
  cat("CV slices:", nrow(cv_h), "\n")

  # step_normalize fits mean and SD on the training partition of each CV fold only.
  # Fitting the scaler on the full training set before CV would leak fold-assessment info.
  svm_recipe <- recipe(
    as.formula(paste(target_col, "~ .")),
    data = train_h
  ) %>%
    step_normalize(all_numeric_predictors())

  svm_spec <- svm_rbf(
    cost      = tune(),
    rbf_sigma = tune()
  ) %>%
    set_engine("kernlab") %>%
    set_mode("regression")

  svm_wf <- workflow() %>%
```

```r
  add_recipe(svm_recipe) %>%
  add_model(svm_spec)

svm_grid <- grid_latin_hypercube(
  cost(),
  rbf_sigma(),
  size = SVM_GRID_SIZE
)

svm_tune <- tune_grid(
  svm_wf,
  resamples = cv_h,
  grid      = svm_grid,
  metrics   = metric_set(rmse, mae, rsq),
  control   = control_grid(verbose = TRUE, save_pred = FALSE)
)

best_svm <- select_best(svm_tune, metric = "rmse")
cat("Best cost:", round(best_svm$cost, 5),
    "| rbf_sigma:", round(best_svm$rbf_sigma, 6), "\n")

final_svm_wf  <- finalize_workflow(svm_wf, best_svm)
final_svm_fit <- fit(final_svm_wf, data = train_h)

svm_preds       <- predict(final_svm_fit, new_data = test_h)$.pred
svm_train_preds <- predict(final_svm_fit, new_data = train_h)$.pred
naive_preds     <- test_df$lag1

svm_metrics_list[[h]] <- compute_metrics(
  test_actual  = test_h[[target_col]],
  test_pred    = svm_preds,
  train_actual = train_h[[target_col]],
  train_pred   = svm_train_preds,
  naive_pred   = naive_preds,
  model        = "SVM",
  horizon      = horizon_label
)
cat("RMSE:", round(svm_metrics_list[[h]]$RMSE, 6),
    "| DA:",  round(svm_metrics_list[[h]]$DA,   4),
    "| Naive DA:", round(svm_metrics_list[[h]]$Naive_DA, 4), "\n")

pred_vs_actual_plot <- ggplot(tibble(Actual = test_h[[target_col]], Predicted = svm_preds), aes(x = A
  geom_point(color = "#141F52", alpha = 0.6) +
  geom_abline(intercept = 0, slope = 1, color = "#E3120B", linetype = "dashed", linewidth = 1) +
  labs(
    title    = "SVM - Predicted vs Actual Returns",
    subtitle = paste0(toupper(horizon_label), " Horizon | Test RMSE: ", round(svm_metrics_list[[h]]$RM
    caption  = "Source: Custom Machine Learning Pipeline. Out-of-sample testing on 20% holdout.",
    x = "Actual Log Return",
    y = "Predicted Log Return"
  ) +
  theme_quant()
```

```r
  ggsave(
    here("paper", "figures", paste0("svm_pred_actual_", horizon_label, ".png")),
    pred_vs_actual_plot,
    width = 8, height = 6, dpi = 300
  )
  cat("Scatter plot saved: paper/figures/svm_pred_actual_", horizon_label, ".png\n", sep = "")

  saveRDS(
    final_svm_fit,
    here("output", "models", paste0("svm_", horizon_label, ".rds"))
  )
  cat("Model saved: output/models/svm_", horizon_label, ".rds\n", sep = "")
}
```

```
##
## SVM | 1D horizon
## CV slices: 19

## Best cost: 0.10419 | rbf_sigma: 0.009853
## RMSE: 0.016341 | DA: 0.32 | Naive DA: 0.2762

## Scatter plot saved: paper/figures/svm_pred_actual_1d.png
## Model saved: output/models/svm_1d.rds
##
## SVM | 3D horizon
## CV slices: 19

## Best cost: 0.03497 | rbf_sigma: 0.009706
## RMSE: 0.026203 | DA: 0.4914 | Naive DA: 0.32

## Scatter plot saved: paper/figures/svm_pred_actual_3d.png
## Model saved: output/models/svm_3d.rds
##
## SVM | 5D horizon
## CV slices: 19

## Best cost: 8.23014 | rbf_sigma: 0.000107
## RMSE: 0.032557 | DA: 0.4762 | Naive DA: 0.3029

## Scatter plot saved: paper/figures/svm_pred_actual_5d.png
## Model saved: output/models/svm_5d.rds
```

```r
svm_metrics <- bind_rows(svm_metrics_list)
```

1.6 6. Artificial Neural Network (nnet — Single Hidden Layer)

The ANN uses `nnet` via tidymodels' `mlp()` interface. Architecture: one hidden layer with a tunable number of hidden units; linear activation on the output node (regression mode). Feature normalisation is applied via the same `step_normalize()` recipe as SVM, ensuring stable gradient descent and comparable input magnitudes.

We tune two parameters:

Parameter	Controls
`hidden_units`	Width of the single hidden layer
`penalty`	L2 weight decay; prevents overfitting

Parameter	Controls

Architectural note: `nnet` uses logistic (sigmoid) activation on hidden units and does not natively support ReLU activation or dropout. The `penalty` (weight decay) parameter provides regularisation analogous to dropout by penalising large weight magnitudes.

```r
set.seed(RANDOM_SEED)

ann_metrics_list <- vector("list", length(TARGET_COLS))

for (h in seq_along(TARGET_COLS)) {

  target_col    <- TARGET_COLS[h]
  horizon_label <- HORIZON_LABS[h]

  cat("\nANN |", toupper(horizon_label), "horizon\n")

  train_h <- train_df %>% select(all_of(c(target_col, PREDICTOR_COLS)))
  test_h  <- test_df  %>% select(all_of(c(target_col, PREDICTOR_COLS)))

  cv_h <- rolling_origin(
    data       = train_h,
    initial    = CV_INITIAL,
    assess     = CV_ASSESS,
    skip       = CV_SKIP,
    cumulative = TRUE
  )
  cat("CV slices:", nrow(cv_h), "\n")

  # Normalisation ensures stable gradient descent across features with different scales
  ann_recipe <- recipe(
    as.formula(paste(target_col, "~ .")),
    data = train_h
  ) %>%
    step_normalize(all_numeric_predictors())

  ann_spec <- mlp(
    hidden_units = tune(),
    penalty      = tune(),
    epochs       = ANN_EPOCHS
  ) %>%
    set_engine("nnet") %>%
    set_mode("regression")

  ann_wf <- workflow() %>%
    add_recipe(ann_recipe) %>%
    add_model(ann_spec)

  ann_grid <- grid_latin_hypercube(
    hidden_units(range = c(ANN_HIDDEN_MIN, ANN_HIDDEN_MAX)),
    penalty(range = c(-5, 0)),   # log10 scale: 1e-5 to 1.0
    size = ANN_GRID_SIZE
  )
```

```r
ann_tune <- tune_grid(
  ann_wf,
  resamples = cv_h,
  grid      = ann_grid,
  metrics   = metric_set(rmse, mae, rsq),
  control   = control_grid(verbose = TRUE, save_pred = FALSE)
)

best_ann <- select_best(ann_tune, metric = "rmse")
cat("Best hidden_units:", best_ann$hidden_units,
    "| penalty:", round(best_ann$penalty, 6), "\n")

final_ann_wf  <- finalize_workflow(ann_wf, best_ann)
final_ann_fit <- fit(final_ann_wf, data = train_h)

ann_preds       <- predict(final_ann_fit, new_data = test_h)$.pred
ann_train_preds <- predict(final_ann_fit, new_data = train_h)$.pred
naive_preds     <- test_df$lag1

ann_metrics_list[[h]] <- compute_metrics(
  test_actual  = test_h[[target_col]],
  test_pred    = ann_preds,
  train_actual = train_h[[target_col]],
  train_pred   = ann_train_preds,
  naive_pred   = naive_preds,
  model        = "ANN",
  horizon      = horizon_label
)
cat("RMSE:", round(ann_metrics_list[[h]]$RMSE, 6),
    "| DA:",  round(ann_metrics_list[[h]]$DA,   4),
    "| Naive DA:", round(ann_metrics_list[[h]]$Naive_DA, 4), "\n")

pred_vs_actual_plot <- ggplot(tibble(Actual = test_h[[target_col]], Predicted = ann_preds), aes(x = A
  geom_point(color = "#141F52", alpha = 0.6) +
  geom_abline(intercept = 0, slope = 1, color = "#E3120B", linetype = "dashed", linewidth = 1) +
  labs(
    title    = "ANN - Predicted vs Actual Returns",
    subtitle = paste0(toupper(horizon_label), " Horizon | Test RMSE: ", round(ann_metrics_list[[h]]$RM
    caption  = "Source: Custom Machine Learning Pipeline. Out-of-sample testing on 20% holdout.",
    x = "Actual Log Return",
    y = "Predicted Log Return"
  ) +
  theme_quant()

ggsave(
  here("paper", "figures", paste0("ann_pred_actual_", horizon_label, ".png")),
  pred_vs_actual_plot,
  width = 8, height = 6, dpi = 300
)
cat("Scatter plot saved: paper/figures/ann_pred_actual_", horizon_label, ".png\n", sep = "")

saveRDS(
  final_ann_fit,
```

```r
    here("output", "models", paste0("ann_", horizon_label, ".rds"))
  )
  cat("Model saved: output/models/ann_", horizon_label, ".rds\n", sep = "")
}
```

```
##
## ANN | 1D horizon
## CV slices: 19

## Best hidden_units: 12 | penalty: 0.72596
## RMSE: 0.016289 | DA: 0.3429 | Naive DA: 0.2762

## Scatter plot saved: paper/figures/ann_pred_actual_1d.png
## Model saved: output/models/ann_1d.rds
##
## ANN | 3D horizon
## CV slices: 19

## Best hidden_units: 7 | penalty: 0.544939
## RMSE: 0.02636 | DA: 0.4648 | Naive DA: 0.32

## Scatter plot saved: paper/figures/ann_pred_actual_3d.png
## Model saved: output/models/ann_3d.rds
##
## ANN | 5D horizon
## CV slices: 19

## Best hidden_units: 3 | penalty: 0.62412
## RMSE: 0.032776 | DA: 0.4762 | Naive DA: 0.3029

## Scatter plot saved: paper/figures/ann_pred_actual_5d.png
## Model saved: output/models/ann_5d.rds
```

```r
ann_metrics <- bind_rows(ann_metrics_list)
```

1.7 7. Results Summary

```r
all_metrics <- bind_rows(svm_metrics, ann_metrics) %>%
  mutate(across(where(is.double), ~ round(., 5)))

kable(
  all_metrics,
  caption   = "SVM and ANN: Out-of-Sample Performance by Horizon",
  col.names = c("Model", "Horizon", "RMSE", "MAE", "R2", "DA", "Naive DA", "Train RMSE", "Train MAE", "T
  digits    = 5
) %>%
  kable_styling(bootstrap_options = c("striped", "condensed"), full_width = FALSE)
```

Table 1: SVM and ANN: Out-of-Sample Performance by Horizon

Model	Horizon	RMSE	MAE	R2	DA	Naive DA	Train RMSE	Train MAE	Train R2	Train D
SVM	1d	0.01634	0.00998	-0.00505	0.32000	0.27619	0.01613	0.00958	0.02435	0.374
SVM	3d	0.02620	0.01903	0.00953	0.49143	0.32000	0.02583	0.01804	0.02167	0.543
SVM	5d	0.03256	0.02473	-0.01350	0.47619	0.30286	0.03245	0.02291	0.03245	0.553
ANN	1d	0.01629	0.00993	0.00130	0.34286	0.27619	0.01619	0.00971	0.01722	0.354

ANN	3d	0.02636	0.01936	-0.00242	0.46476	0.32000	0.02561	0.01808	0.03840	0.535
ANN	5d	0.03278	0.02494	-0.02718	0.47619	0.30286	0.03202	0.02308	0.05783	0.561

1.8 8. Save Outputs

```
write_csv(all_metrics, here("output", "predictions", "svm_ann_metrics.csv"))
message("Metrics saved to output/predictions/svm_ann_metrics.csv")

if (!dir.exists(here("paper", "tables"))) dir.create(here("paper", "tables"))

latex_tbl <- kable(
  all_metrics,
  format    = "latex",
  booktabs  = TRUE,
  escape    = TRUE,
  linesep   = "",
  col.names = c("Model", "Horizon", "RMSE", "MAE", "R2", "DA", "Naive DA", "Train RMSE", "Train MAE", "T
  digits    = 5
)

# Build table manually: footnote outside table sizing
svm_ann_note <- paste0(
  "DA = Directional Accuracy (fraction of test-set observations where the ",
  "predicted sign matches the realised sign). ",
  "Naive DA uses the previous day's return (lag1) as the prediction. ",
  "R2 is out-of-sample and may be negative if the model performs worse than the mean. ",
  "SVM features are normalised inside the workflow (zero mean, unit variance). ",
  "ANN uses logistic hidden-unit activation and L2 weight decay via the nnet engine."
)

latex_tbl <- paste0(
  "\\begin{table}[!ht]\n\\centering\n",
  "\\caption{\\label{tab:svm_ann_metrics}SVM and ANN: Out-of-Sample Performance by Horizon}\n",
  "\\footnotesize\n",
  "\\setlength{\\tabcolsep}{3.5pt}\n",
  as.character(latex_tbl), "\n",
  "\\multicolumn{11}{p{\\linewidth}}{\\rule{0pt}{1em}\\textit{Note.} ", svm_ann_note, "}\\\\\n",
  "\\end{tabular}\n",
  "\\end{table}"
)

writeLines(latex_tbl, here("paper", "tables", "svm_ann_metrics.tex"))
message("LaTeX table saved to paper/tables/svm_ann_metrics.tex")
message("05_models_svm_ann.Rmd: Complete.")
```